

Type Conversion & Type Promotion in Java

Type Conversion & Type Promotion in Java

1. Introduction
2. Implicit vs. Explicit Conversion
3. Widening (Implicit) Conversion
4. Narrowing (Explicit) Conversion — Casting
5. Truncating Conversion (float/double → int)
6. What About `boolean` ?
7. Automatic Type Promotion in Expressions

★ One-Page Revision

📝 Practice Questions

🔥 Quick FAQ

Type Conversion & Type Promotion in Java

Implicit vs. Explicit Casting — Lecture 5

1. Introduction

Java has 8 primitive data types (`byte` , `short` , `int` , `long` , `float` , `double` , `char` , `boolean`), and you'll often need to move a value from one type to another — e.g. storing a `byte` inside an `int` , or a `double` inside an `int` .

This is called **type conversion**, and Java handles it in two very different ways depending on whether data could be **lost** in the process.

2. Implicit vs. Explicit Conversion

	Implicit Conversion	Explicit Conversion
Who does it?	Java does it automatically	You must do it manually
Also called	Widening Conversion	Narrowing Conversion
Rule	Destination type is wider than source	Destination type is smaller than source
Risk of data loss?	No	Yes — requires a cast
Example	<code>byte</code> → <code>int</code>	<code>int</code> → <code>byte</code>

💡 Easy Hinglish Explanation

"Agar chhote dabbe (byte) ki value ko bade dabbe (int) mein daalna hai, to koi problem nahi — sab kuch aasani se fit ho jaayega. Lekin agar bade dabbe ki value ko chhote dabbe mein daalna hai, to kuch cheez chhoot sakti hai — isliye Java khud se nahi karta, tumhe explicitly bolna padta hai 'haan, main jaanta hoon, phir bhi karo.'"

3. Widening (Implicit) Conversion

Rule: The destination data type must be **wider** (able to hold a bigger range) than the source.

Example:

```
byte b = 24;
int i = b;           // works automatically – no cast needed

System.out.println(i); // Output: 24
```

Why it works: `byte` is 8 bits (range -128 to 127); `int` is 32 bits with a much bigger range. Since anything a `byte` can hold, an `int` can also hold, Java converts it safely on its own.

📌 **Important:** This also applies to `char` → `int`. Internally, every `char` is stored as its **Unicode number** (e.g. `'a'` = 97), so widening a `char` to `int` just reveals that number:

```
char c = 'a';
int i = c;
System.out.println(i); // Output: 97
```

4. Narrowing (Explicit) Conversion — Casting

Rule: Converting from a **wider** type to a **smaller** one requires an explicit **cast**, because data might not fit — and Java refuses to do this silently.

Syntax: `destinationType variable = (destinationType) sourceValue;`

```
int i = 300;
byte b = (byte) i;      // must cast - int is wider than byte

System.out.println(b); // Output: 44 (NOT 300!)
```

Why does 300 become 44 ?

Step 1 — Convert 300 to binary (32-bit int):

```
300 → 00000000 00000000 00000001 00101100
```

Step 2 — Keep only the last 8 bits (byte can only hold 8 bits):

```
00101100 → decimal 44
```

★ **Exam Tip — Shortcut**

Instead of doing full binary math, you can just take `value % range_of_target_type` : `300 % 256 = 44` (since a byte has $2^8 = 256$ possible values)

⚠ **Common Mistake**

This bit-truncation is called **data truncation** — it silently produces a *wrong-looking but technically valid* number. Always check that your value actually fits the target type's range before narrowing, or you'll get unexpected results like this.

5. Truncating Conversion (float/double → int)

A special case of narrowing: converting a **decimal** type to a **whole-number** type.

```
float f = 615.678f;
int i = (int) f;

System.out.println(i);    // Output: 615
```

 **Important:** Casting `float` / `double` to `int` doesn't round — it simply **chops off everything after the decimal point**. `615.678` becomes `615`, not `616`.

⚠ Common Mistake

Even though `float` and `int` are both 32 bits, you **still** need an explicit cast between them — same bit-width doesn't mean same data can always fit (a `float`'s decimal part has nowhere to go in an `int`).

6. What About `boolean` ?

⚠ `boolean` cannot be converted to or from any other type — not even with a cast.

```
boolean bool = false;
int i = bool;           // ❌ Compile error: type mismatch
int i = (int) bool;     // ❌ Still an error: cannot cast boolean to int
```

Why: `boolean` isn't a number at all — it's a completely separate, special data type in Java with only two possible values (`true` / `false`). There's no numeric equivalent to convert to.

7. Automatic Type Promotion in Expressions

Even without writing any cast yourself, Java automatically **promotes** smaller types during calculations — otherwise, simple math would constantly overflow.

The Problem:

```
byte a = 50, b = 40, c = 100;
int i = a * b / c;    // Works fine!
```

`a * b` = `50 × 40` = `2000` — already outside `byte`'s range (−128 to 127)! If Java tried to keep this as a `byte`, it couldn't even hold the intermediate result. So Java **automatically promotes** `a`, `b`, and `c` to

`int` just for this calculation.

The 4 Rules of Type Promotion

Rule	What Happens
1	<code>byte</code> , <code>short</code> , and <code>char</code> are always promoted to <code>int</code> in any calculation
2	If one operand is <code>long</code> , the entire expression becomes <code>long</code>
3	If one operand is <code>float</code> , the entire expression becomes <code>float</code>
4	If one operand is <code>double</code> , the entire expression becomes <code>double</code>

💡 Easy Hinglish Explanation

"Simple sa rule hai — jo bhi sabse bada data type expression mein maujood hai, poora expression usi mein convert ho jaata hai. Chhote types (byte, short, char) to calculation shuru hote hi automatically int ban jaate hain."

The Confusing Case: `b = b * 2`

```
byte b = 50;
b = b * 2;    // ❌ Compile error: cannot convert from int to byte
```

Why does this fail even though there's no `int` anywhere in the code?

Because `b * 2` triggers **Rule 1** — `b` gets promoted to `int` for the multiplication, so `b * 2` produces an `int` result (`100`). You're then trying to assign that `int` back into a `byte` variable — which needs an explicit cast:

```
b = (byte) (b * 2);    // ✅ Works – explicit cast fixes it
```

⚠️ Common Mistake

Writing `b = (byte) b * 2` (without the extra parentheses) casts *only* `b`, not the whole expression `b * 2` — the multiplication still happens in `int`, and you'll get the same error. Wrap the **entire expression** in parentheses before casting it.

★ One-Page Revision

Core Definitions

- **Implicit (Widening) Conversion** = Java does it automatically; destination type is wider than source
- **Explicit (Narrowing) Conversion** = You must cast manually; destination type is smaller than source
- **Truncating Conversion** = casting a decimal type to a whole-number type; chops the decimal part (no rounding)
- **Type Promotion** = automatic upgrade of `byte` / `short` / `char` to `int` (or higher) during calculations

Key Rules

- Widening: `byte → short → int → long → float → double` (no cast needed)
- Narrowing: reverse direction — always needs `(type)` cast
- `char → int` is widening (reveals the Unicode number)
- `boolean` can **never** convert to/from any other type, cast or not
- Shortcut for narrowing integers: `value % range_of_target_type`
- Type promotion rules: `byte/short/char → int` always; then `int → long → float → double` (highest type wins)

Cast Syntax

```
destinationType variable = (destinationType) sourceValue;
```



Practice Questions

Basic

1. What's the difference between widening and narrowing conversion?
2. Why can't `boolean` be cast to `int`?

Medium 3. What will `byte b = (byte) 300;` print, and why?

Interview 4. Explain, with binary, why casting `300` to a `byte` produces `44`. 5. What are the 4 rules of automatic type promotion in Java expressions?



Quick FAQ

Does casting round decimal numbers? No — `(int) 615.678` gives `615`, not `616`. It truncates, it doesn't round.

Is `float → int` conversion widening or narrowing? Narrowing — even though both are 32-bit, `int` can't store the decimal part, so an explicit cast is still required.